

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«ВОЗРАСТ В НЕОЖИДАННЫХ ЕДИНИЦАХ»
Вариант 20

Студент:

Я.Н. Ковалевский

Преподаватель:

Д.В. Куприянова

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Освоить ключевые возможности среды разработки IntelliJ IDEA, применяемые при создании объектно-ориентированных приложений на языке Java: редактирование, сборку, запуск и отладку программ с графическим интерфейсом. Практически применить архитектурный паттерн MVC с активной моделью.

В соответствии с вариантом 20 задания необходимо разработать утилиту для расчёта возраста человека в неожиданных единицах измерения. Для выполнения работы поставлены следующие задачи:

1. Развернуть и настроить Java-проект в IntelliJ IDEA с применением системы сборки Gradle и библиотеки JavaFX, а также освоить инструменты редактирования, сборки, запуска и отладки;
2. Определить, какие средства IntelliJ IDEA подходят для работы с UML и SysML v2, и зафиксировать результат настройки в отчёте;
3. Спроектировать графическое приложение, содержащее главную форму и отдельное модальное окно для ввода даты рождения;
4. Реализовать ввод даты рождения без стандартного поля выбора даты, а через отдельные текстовые поля для дня, месяца и года, а также обеспечить сохранение последних корректно введённых значений и их восстановление при повторном открытии окна ввода;
5. Добавить проверку входных данных на допустимые диапазоны дня, месяца и года, на существование даты в григорианском календаре и на отсутствие даты в будущем, а также вывод понятного сообщения при ошибке;
6. Разделить приложение на модель, представление и контроллер, обеспечив активное уведомление представления со стороны модели через интерфейс `ModelListener`;
7. Протестировать основные сценарии работы программы, включая корректный ввод, ввод некорректных данных и восстановление последних значений, и подготовить ответы на контрольные вопросы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Интегрированная среда разработки и её компоненты

Среда разработки объединяет в себе редактор кода, инструменты навигации и рефакторинга, систему сборки, средства запуска, отладчик, терминал и поддержку контроля версий. IntelliJ IDEA выполняет анализ Java-кода прямо в процессе редактирования, подсвечивает ошибки и предупреждения, предлагает быстрые исправления и даёт возможность запускать класс с методом main, не покидая редактор.

2.2 Редактирование и навигация в IntelliJ IDEA

Редактор IntelliJ IDEA предоставляет автодополнение, автоматический импорт классов, форматирование, безопасное переименование и поиск использований. Переход к объявлению выполняется по Ctrl+B, поиск использований – по Alt+F7, форматирование – по Ctrl+Alt+L, а вызов намерений и быстрых исправлений – по Alt+Enter. Дерево проекта отображается в окне Project, а сообщения компилятора и результаты работы программы – в окнах Build и Run.

2.3 Сборка, запуск и отладка Java-приложения

Компилятор javac преобразует исходные файлы в байт-код для JVM. В IntelliJ IDEA сборка проекта выполняется командой Build Project, а запуск и отладка конфигурации класса App – командами Run и Debug. Точка останова ставится щелчком слева от строки или сочетанием Ctrl+F8. При остановке в окне Debug отображаются стек вызовов и текущие значения переменных. Пошаговое выполнение обеспечивают команды Step Over, Step Into и Step Out.

2.4 Паттерн MVC

Паттерн MVC делит приложение на три составляющие. Model хранит состояние и реализует предметную логику. View отвечает за интерфейс и отображение данных. Controller обрабатывает действия пользователя и передаёт команды модели. Подобное разделение снижает связанность классов и позволяет тестировать логику независимо от графического интерфейса.

2.5 Активная и пассивная модель

При пассивной модели контроллер после изменения данных сам вызывает обновление представления. Активная модель, напротив, сама уведомляет подписчиков об изменении состояния. В разработанном приложении AgeModel реализует паттерн «Наблюдатель» через интерфейс ModelListener.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработана утилита, принимающая дату рождения в виде трёх отдельных чисел – дня, месяца и года – и вычисляющая возраст человека в нескольких единицах измерения: годах, месяцах, днях и минутах. На главной форме расположена кнопка «Ввести данные», по нажатию на которую открывается модальное окно `InputDialog`. Ввод даты организован без стандартного поля выбора даты. Приложение поддерживает восстановление введённых данных: при повторном открытии окна ввода в поля автоматически подставляются последние успешно обработанные значения.

3.2 Настройка среды и расширений для UML и SysML v2

В IntelliJ IDEA создан проект с каталогами `src/main/java` и `src/main/resources`; основной пакет – `by.bsuir.agecalc`. В качестве Project SDK выбран JDK 21. Класс `by.bsuir.agecalc.Main` содержит метод `start` и используется в конфигурации запуска. Сборка выполняется через Gradle, для графического интерфейса подключена библиотека JavaFX 21. Для работы с UML установлен плагин `PlantUML Integration`, диаграмма хранится в файле `docs/mvc.puml`. Для материалов SysML v2 применяется текстовая нотация языка и совместимое средство подсветки, установленное через `Settings – Plugins`.

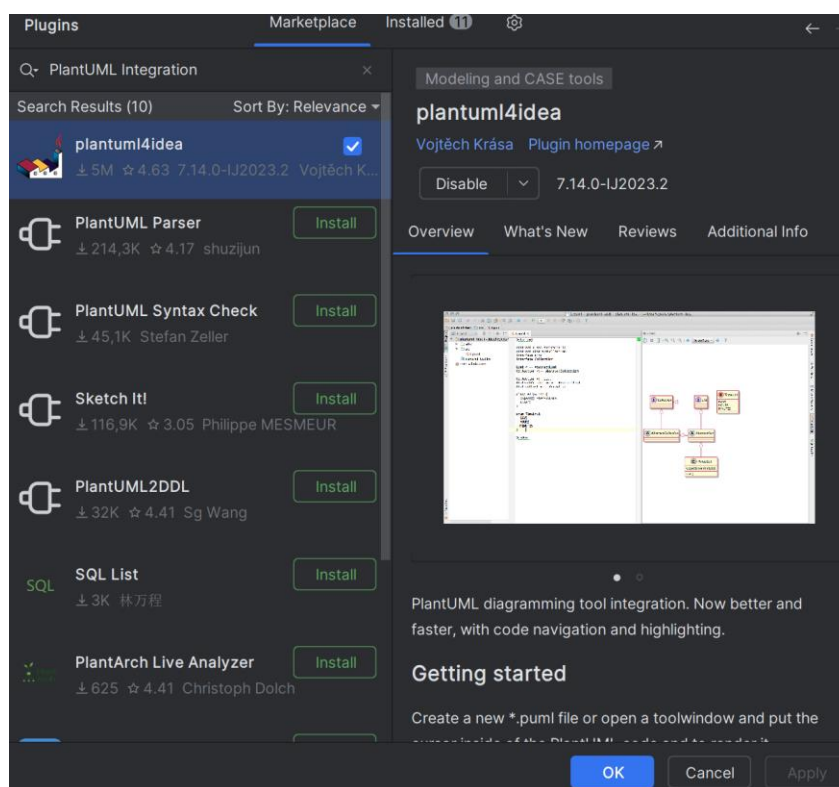


Рисунок 3.1 – Средства IntelliJ IDEA для работы с UML и SysML v2

Структурное представление системы в нотации, близкой к SysML v2:

```
package AgeCalculator {
    part def AgeModel;
    part def MainView;
    part def InputDialog;
    part def MainController;

    part application {
        part model : AgeModel;
        part view : MainView;
        part dialog : InputDialog;
        part controller : MainController;
        connect view to controller;
        connect controller to model;
        connect model to view;
    }
}
```

3.3 Демонстрация отладчика

Работа отладчика демонстрируется в методе `calculateAge` класса `AgeModel`. Точка останова устанавливается на строке вычисления переменной `totalMinutes`. В окне `Debug` можно проследить, как значения `birthDate` и `now` влияют на итоговый результат, а также проверить точность промежуточных вычислений с помощью функции `Evaluate Expression`.

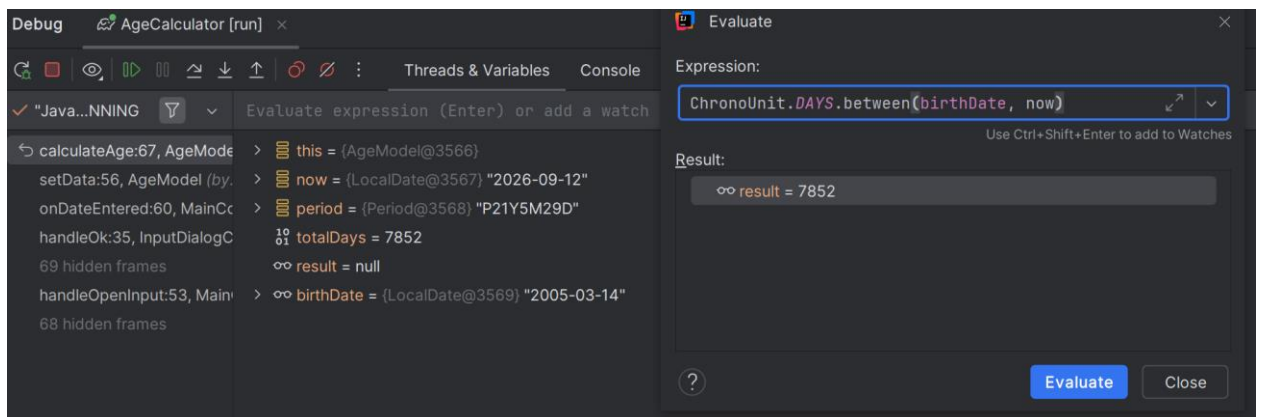


Рисунок 3.2 – Остановка программы в отладчике IntelliJ IDEA

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на следующие компоненты:

- `AgeModel` – хранит дату рождения и результат расчёта, содержит бизнес-логику вычисления возраста и механизм уведомления слушателей через интерфейс `ModelListener`;
- `MainController` – реализует `AgeModel.ModelListener` для автоматического обновления главного окна при изменении данных, управляет

жизненным циклом модального окна ввода и обрабатывает ввод пользователя;

- `InputDialogController` – обрабатывает действия пользователя в модальном окне: выполняет парсинг строк в числа, проверяет корректность и передаёт данные в `MainController`;
- `MainView.fxml` – главный интерфейс приложения с кнопкой «Ввести данные», полями для отображения даты рождения и результата;
- `InputDialog.fxml` – модальное окно для сбора даты рождения, состоящее из трёх текстовых полей: день, месяц, год;
- `Main` – точка входа, инициализирующая JavaFX-приложение и загружающая главную форму.

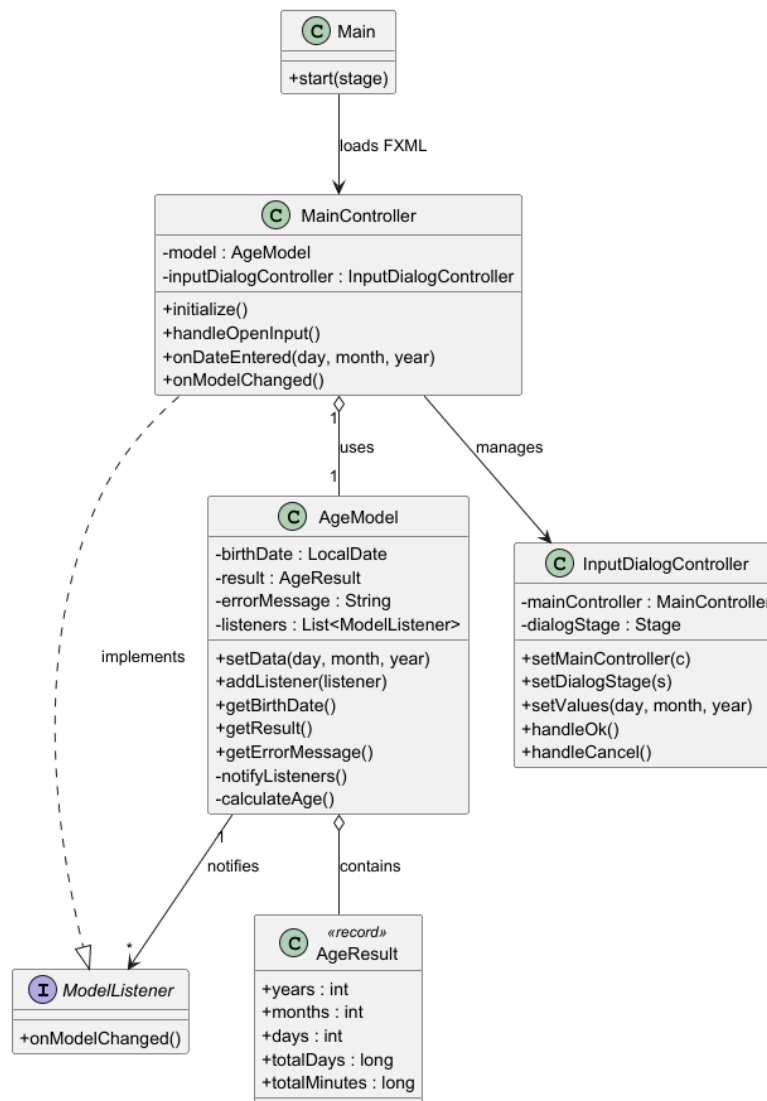


Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

3.5 Модель

`AgeModel` представляет собой активную модель. Метод `setData` выполняет многоуровневую проверку: сначала контролируются диапазоны (месяц 1–12, день 1–31, год 1900–текущий), затем проверяется фактическое

существование даты через `LocalDate.of()` и отсутствие даты в будущем. После успешной валидации вызывается `calculateAge()`, которая вычисляет возраст с использованием классов `Period` и `ChronoUnit`.

3.6 Представление

`MainView.fxml` отображает введенную дату рождения и рассчитанный возраст в годах, месяцах, днях и минутах. Благодаря реализации интерфейса `ModelListener` метод `onModelChanged` обновляет текстовые метки мгновенно после того, как модель зафиксировала изменения. `InputDialog.fxml` обеспечивает ввод даты через три отдельных текстовых поля с поддержкой клавиши `Enter` для подтверждения ввода.

3.7 Контроллер

`MainController` управляет жизненным циклом `InputDialog` и отвечает за передачу данных из окна ввода в модель. `InputDialogController` выполняет безопасный перевод пользовательского ввода из текстового формата в числовой. В случае `NumberFormatException` контроллер перехватывает ошибку и выводит её пользователю через метку `lblDialogError` в окне ввода. Ошибки валидации, сформированные в модели, отображаются в главном окне и дублируются через `Alert`.

3.8 Запоминание данных и обработка ошибок

Приложение предотвращает ввод некорректных дат. Сообщения об ошибках (например, «Указанной даты не существует в григорианском календаре!») формируются в модели и передаются через контроллер во `View`. Восстановление данных реализовано через метод `setValues()`, который вызывается перед показом окна и подставляет последние успешно обработанные значения.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В IntelliJ IDEA создано и проверено Java-приложение по варианту 20. Программа вычисляет возраст человека в годах, месяцах, днях и минутах, сохраняет последнее успешное состояние и сообщает о некорректном вводе. Архитектура соответствует MVC с активной моделью: контроллер обрабатывает пользовательское действие, модель выполняет вычисления и уведомляет представление через интерфейс ModelListener.

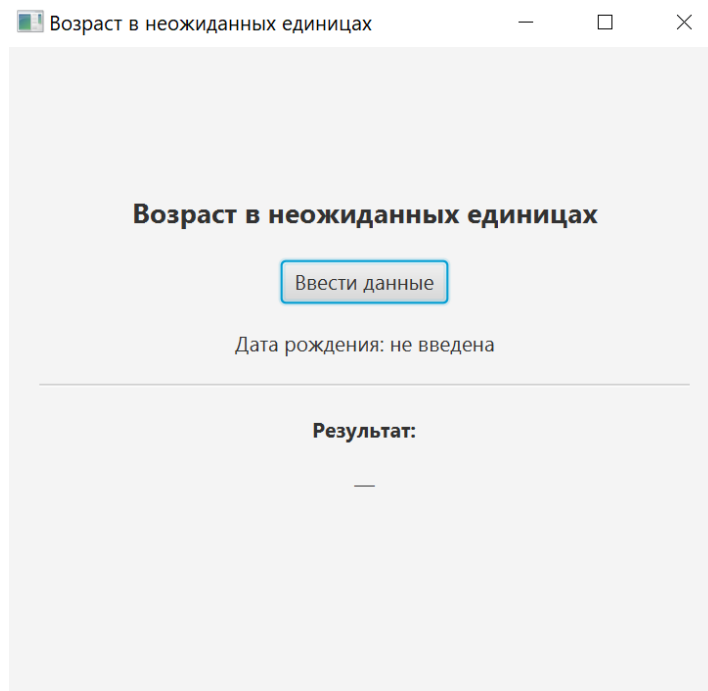


Рисунок 4.1 – Главное окно

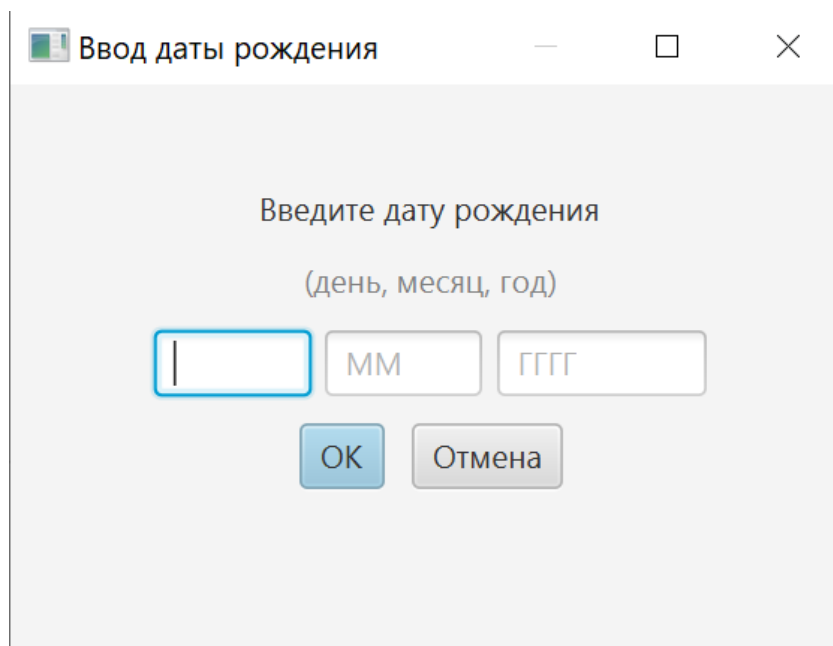


Рисунок 4.2 – Окно ввода данных

Возраст в неожиданных единицах

Ввести данные

Дата рождения: 1993-02-16

Результат:

Лет: 33
Месяцев: 6
Дней: 27

Всего дней: 12261
Всего минут: 17 655 840

Рисунок 4.3 – Результат вычисления

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

Интегрированная среда разработки объединяет редактор исходного кода, средства навигации и рефакторинга, систему сборки, механизмы запуска и отладки, терминал и модуль работы с системами контроля версий. Все эти инструменты доступны из одного окна, что избавляет разработчика от необходимости переключаться между разными приложениями.

Вопрос 2 Каковы основные приёмы работы с редактором кода?

Работа с редактором кода строится на автодополнении, автоматическом подключении нужных классов, форматировании, безопасном переименовании сущностей и поиске всех мест их использования. Перемещение по коду ускоряют горячие клавиши: переход к объявлению, поиск использований, вызов быстрых исправлений и намерений. Сообщения компилятора и результаты работы программы выводятся в отдельных окнах Build и Run.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

Сборка проекта запускается командой Build Project, в ходе которой компилятор `javac` превращает исходные тексты в байт-код для виртуальной машины JVM. Запуск и отладка выполняются через конфигурацию, указывающую главный класс приложения. Точка останова ставится щелчком в левом поле редактора, после чего окно Debug показывает стек вызовов и текущие значения переменных, а команды Step Over, Step Into и Step Out позволяют продвигаться по программе пошагово.

Вопрос 4 Что такое MVC?

Паттерн MVC разделяет приложение на три обособленные части. Модель хранит состояние и отвечает за предметную логику, представление строит интерфейс и выводит данные, а контроллер принимает действия пользователя и превращает их в команды для модели. Такое разделение ослабляет связи между классами и позволяет проверять логику независимо от графической оболочки.

Вопрос 5 Чем отличается активная модель от пассивной?

Разница между активной и пассивной моделью проявляется в том, кто инициирует обновление представления. При пассивной модели это делает контроллер – уже после того, как данные изменились. Активная модель, напротив, сама оповещает подписчиков о любом изменении своего состояния, реализуя паттерн «Наблюдатель», благодаря чему представление получает уведомления автоматически.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности IntelliJ IDEA для редактирования, сборки, запуска и отладки Java-приложения. Рассмотрены средства работы с UML и SysML v2, подготовлена диаграмма классов приложения.

В соответствии с выданным вариантом разработана утилита для расчёта возраста в неожиданных единицах измерения с графическим интерфейсом на базе JavaFX. Программа принимает дату рождения через отдельные поля для дня, месяца и года, выполняет проверку корректности введённых данных и сохраняет последнее успешное состояние. Компоненты разделены по паттерну MVC, а активная модель уведомляет представление об изменениях через интерфейс ModelListener.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг исходного кода приложения

InputDialogController.java

```
package by.bsuir.agecalc.controller;

import javafx.fxml.FXML;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.stage.Stage;

public class InputDialogController {
    @FXML private TextField txtDay;
    @FXML private TextField txtMonth;
    @FXML private TextField txtYear;
    @FXML private Label lblDialogError;

    private MainController mainController;
    private Stage dialogStage;

    public void setMainController(MainController c) { this.mainController = c; }
    public void setDialogStage(Stage s) { this.dialogStage = s; }
    public Stage getDialogStage() { return dialogStage; }

    public void setValues(Integer day, Integer month, Integer year) {
        txtDay.setText(day != null ? String.valueOf(day) : "");
        txtMonth.setText(month != null ? String.valueOf(month) : "");
        txtYear.setText(year != null ? String.valueOf(year) : "");
        lblDialogError.setText("");
    }

    @FXML
    private void handleOk() {
        try {
            int day = Integer.parseInt(txtDay.getText().trim());
            int month = Integer.parseInt(txtMonth.getText().trim());
            int year = Integer.parseInt(txtYear.getText().trim());
            mainController.onDateEntered(day, month, year);
            dialogStage.close();
        } catch (NumberFormatException e) {
            lblDialogError.setText("Введите целые числа");
        }
    }

    @FXML
    private void handleCancel() {
        dialogStage.close();
    }
}
```

MainController.java

```
package by.bsuir.agecalc.controller;

import by.bsuir.agecalc.model.AgeModel;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
```

```

import javafx.scene.control.Alert;
import javafx.scene.control.Label;
import javafx.stage.Modality;
import javafx.stage.Stage;

import java.io.IOException;
import java.time.LocalDate;

public class MainController implements AgeModel.ModelListener {
    @FXML private Label lblBirthDate;
    @FXML private Label lblResult;
    @FXML private Label lblError;

    private final AgeModel model = new AgeModel();
    private InputDialogController inputDialogController;

    @FXML
    public void initialize() {
        model.addListener(this);
    }

    @FXML
    private void handleOpenInput() {
        try {
            if (inputDialogController == null) {
                FXMLLoader loader = new FXMLLoader(getClass().getResource(
                    "/view/InputDialog.fxml"));
                Stage dialog = new Stage();
                dialog.initModality(Modality.APPLICATION_MODAL);
                dialog.setTitle("Ввод даты рождения");
                dialog.setScene(new Scene(loader.load()));

                inputDialogController = loader.getController();
                inputDialogController.setMainController(this);
                inputDialogController.setDialogStage(dialog);
            }

            LocalDate last = model.getBirthDate();
            if (last != null) {
                inputDialogController.setValues(
                    last.getDayOfMonth(), last.getMonthValue(),
last.getYear());
            } else {
                inputDialogController.setValues(null, null, null);
            }

            inputDialogController.getDialogStage().showAndWait();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public void onDateEntered(int day, int month, int year) {
        model.setData(day, month, year);
    }

    @Override
    public void onModelChanged() {
        String err = model.getErrorMessage();
        if (err != null) {
            lblError.setText(err);
            lblResult.setText("-");
            Alert alert = new Alert(Alert.AlertType.ERROR);
            alert.setTitle("Ошибка");

```

```

        alert.setHeaderText(null);
        alert.setContentText(err);
        alert.showAndWait();
        return;
    }
    lblError.setText("");
    LocalDate bd = model.getBirthDate();
    if (bd != null) {
        lblBirthDate.setText("Дата рождения: " + bd);
    }
    AgeModel.AgeResult r = model.getResult();
    if (r != null) {
        lblResult.setText(String.format(
            "Лет: %d\nМесяцев: %d\nДней: %d\n\nВсего дней: %d\nВсего
минут: %,d",
            r.years(), r.months(), r.days(), r.totalDays(),
            r.totalMinutes()));
    }
}
}

```

AgeModel.java

```

package by.bsuir.agecalc.model;

import java.time.LocalDate;
import java.time.Period;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;

public class AgeModel {
    public interface ModelListener {
        void onModelChanged();
    }

    private LocalDate birthDate;
    private AgeResult result;
    private String errorMessage;
    private final List<ModelListener> listeners = new ArrayList<>();

    public void addListener(ModelListener listener) {
        listeners.add(listener);
    }

    private void notifyListeners() {
        for (ModelListener l : listeners) l.onModelChanged();
    }

    public void setData(int day, int month, int year) {
        errorMessage = null;
        result = null;

        if (month < 1 || month > 12) {
            errorMessage = "Месяц должен быть в диапазоне 1-12";
            notifyListeners();
            return;
        }
        if (day < 1 || day > 31) {
            errorMessage = "День должен быть в диапазоне 1-31";
            notifyListeners();
            return;
        }
    }
}

```

```

        if (year < 1900 || year > LocalDate.now().getYear()) {
            errorMessage = "Год должен быть в диапазоне 1900-" +
LocalDate.now().getYear();
            notifyListeners();
            return;
        }

        try {
            LocalDate date = LocalDate.of(year, month, day);
            if (date.isAfter(LocalDate.now())) {
                errorMessage = "Дата рождения не может быть в будущем";
                notifyListeners();
                return;
            }
            this.birthDate = date;
            calculateAge();
        } catch (java.time.DateTimeException e) {
            errorMessage = "Указанной даты не существует в григорианском
календаре!";
        }
        notifyListeners();
    }

    private void calculateAge() {
        LocalDate now = LocalDate.now();
        Period period = Period.between(birthDate, now);
        long totalDays = ChronoUnit.DAYS.between(birthDate, now);
        long totalMinutes = ChronoUnit.MINUTES.between(
            birthDate.atStartOfDay(), now.atStartOfDay());

        result = new AgeResult(
            period.getYears(),
            period.getMonths(),
            period.getDays(),
            totalDays,
            totalMinutes
        );
    }

    public LocalDate getBirthDate() { return birthDate; }
    public AgeResult getResult() { return result; }
    public String getErrorMessage() { return errorMessage; }

    public record AgeResult(int years, int months, int days,
        long totalDays, long totalMinutes) {}
}

```

Main.java

```

package by.bsuir.agecalc;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class Main extends Application {
    @Override
    public void start(Stage stage) throws Exception {
        FXMLLoader loader = new FXMLLoader(
            Main.class.getResource("/view/MainView.fxml"));
        Scene scene = new Scene(loader.load(), 420, 380);
        stage.setTitle("Возраст в неожиданных единицах");
    }
}

```

```

        stage.setScene(scene);
        stage.show();
    }
}

```

InputDialog.fxml

```

<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.geometry.Insets?>

<VBox xmlns="http://javafx.com/javafx/21"
      xmlns:fx="http://javafx.com/fxml/1"
      fx:controller="by.bsuir.agecalc.controller.InputDialogController"
      alignment="CENTER" spacing="10" prefWidth="320" prefHeight="220">
    <padding><Insets top="15" right="15" bottom="15" left="15"/></padding>

    <Label text="Введите дату рождения"/>
    <Label text="(день, месяц, год)" style="-fx-text-fill: gray;"/>

    <HBox spacing="5" alignment="CENTER">
        <TextField fx:id="txtDay" promptText="ДД" prefWidth="60"/>
        <TextField fx:id="txtMonth" promptText="ММ" prefWidth="60"/>
        <TextField fx:id="txtYear" promptText="ГГГГ" prefWidth="80"/>
    </HBox>

    <HBox spacing="10" alignment="CENTER">
        <Button text="OK" onAction="#handleOk" defaultButton="true"/>
        <Button text="Отмена" onAction="#handleCancel" cancelButton="true"/>
    </HBox>

    <Label fx:id="lblDialogError" style="-fx-text-fill: red;"/>
</VBox>

```

MainView.fxml

```

<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.geometry.Insets?>

<VBox xmlns="http://javafx.com/javafx/21"
      xmlns:fx="http://javafx.com/fxml/1"
      fx:controller="by.bsuir.agecalc.controller.MainController"
      alignment="CENTER" spacing="15" prefWidth="420" prefHeight="380">
    <padding><Insets top="20" right="20" bottom="20" left="20"/></padding>

    <Label text="Возраст в неожиданных единицах"
          style="-fx-font-size: 16px; -fx-font-weight: bold;"/>

    <Button text="Ввести данные" onAction="#handleOpenInput"/>

    <Label fx:id="lblBirthDate" text="Дата рождения: не введена"/>

    <Separator/>

    <Label text="Результат:" style="-fx-font-weight: bold;"/>
    <Label fx:id="lblResult" text="-" wrapText="true"/>

    <Label fx:id="lblError" style="-fx-text-fill: red;" wrapText="true"/>
</VBox>

```